Discharge Inception

Release 1.0

André Kapelrud, Robert Marskar

Mar 11, 2026

CONTENTS

1 Installation **3**
- 1.1 Prerequisites . . . 3
- 1.2 Get the source . . . 4
- 1.3 Installation instructions . . . 4
- 1.4 SLURM configuration . . . 5
- 1.5 The CLI . . . 6

2 Architecture **11**
- 2.1 Overview . . . 11
- 2.2 Repository layout . . . 11
- 2.3 Databases and studies . . . 12
- 2.4 Run definition . . . 12
- 2.5 Defining parameter spaces . . . 13
- 2.6 JSON URI syntax . . . 14
- 2.7 Dummy parameters . . . 16
- 2.8 Output directory structure . . . 16
- 2.9 The full call chain . . . 17
- 2.10 Script roles . . . 18

3 Jobscripts guide **19**
- 3.1 Setting up a simulation directory . . . 19
- 3.2 Where jobscripts fit . . . 21
- 3.3 GenericArrayJob.sh — the SLURM wrapper . . . 21
- 3.4 Specifying the inception database options . . . 22
- 3.5 Specifying the plasma simulation options . . . 23

4 Post-processing **25**
- 4.1 Extracting partial discharge inception voltages . . . 25
- 4.2 Identifying discharge inception events from plasma logs . . . 27
- 4.3 Analysing simulation time-series output . . . 28
- 4.4 Plotting relative electric field change over time . . . 30

5 Example — Rod Case **33**
- 5.1 Prerequisites . . . 33
- 5.2 Compiling the executable . . . 33
- 5.3 Running a smoke test . . . 34
- 5.4 Inspecting the parameter space definition . . . 34
- 5.5 Running the configurator . . . 36
- 5.6 Output directory layout . . . 36
- 5.7 Monitoring jobs . . . 37

5.8 Inspecting results 38

Index **39**

Documentation for a framework of Python and SLURM scripts for running chombo-discharge studies over wide parameter spaces. This program provides an interface for coupling two different modules within chombo-discharge:

- DischargeInceptionStepper, which can compute values of the ionization integral for a provided geometry and gas.
- ItoKMC, which can run plasma simulations.

The interface consists of parametric sweeps where the partial discharge inception voltage (PDIV) is first computed using DischargeInceptionStepper by adjusting one or more free parameters. This first step builds a database of the PDIV as a function of the various free parameters. In a second step, the plasma solver is run for each unique set of the free parameters, and for a specified range of voltages. The intention of this composite procedure is that users will quickly be able to:

1. Easily generate PDIV-vs-parameter databases.
2. Run multiple plasma simulations corresponding to these databases to provide even more accurate information on the discharge processes themselves.

The second step provides a self-consistent coupling to the electric field, accounts for space-charge screening, formative time lag, and so on. This step is automatic run by submitting jobs via a SLURM queue system, so the user does not have to be directly involved in the logistics of setting up these simulations.

CHAPTER

ONE

INSTALLATION

1.1 Prerequisites

1.1.1 Python

`discharge-inception` requires **Python >= 3.10**. The minimum version is set by the use of structural pattern matching (`match`/`case`) in the parameter handling code and by `tomllib`, which entered the standard library in 3.11 but is available as a backport (`tomli`) for 3.10. Using the newest Python 3.x release available on your cluster is recommended.

`numpy` is a declared dependency and is installed automatically by `pip`. Certain post-processing scripts additionally require `matplotlib` and `scipy`, available via the `[plot]` extra:

```
pip install -e ".[plot]"
```

1.1.2 chombo-discharge

A compiled chombo-discharge installation is required to build and run the simulation executables inside `Exec/`. The framework itself (the Python package and SLURM scripts) does not link against chombo-discharge, but every `Exec/` case depends on it at compile time.

The environment variable `DISCHARGE_HOME` must be set and exported before compiling any `Exec/` case. It should point to the root of the chombo-discharge source tree — the same directory that contains `GNUmakefile.defs`. Setting this in your `.bashrc` (or the cluster environment module) is the most convenient approach.

1.1.3 SLURM

SLURM client tools (`sbatch`, `squeue`, `sinfo`, `scancel`) must be available on the machine from which you submit jobs. On HPC clusters these are typically pre-installed by system administrators — you need only an account and partition access.

The framework submits jobs programmatically: the `discharge-inception run` command calls `sbatch` directly from Python, so the SLURM client must be on the `PATH` of the machine where you run the configurator. Compute nodes themselves do not need to call `sbatch` unless you use the two-stage pipeline, in which case `PlasmaJobscript.py` submits a child array from inside the first SLURM job.

For local testing, both the SLURM controller (`slurmctld`) and the worker daemon (`slurmd`) can run on the same workstation or WSL instance. See the SLURM documentation for installation instructions.

1.2 Get the source

Clone the repository from GitHub. The clone contains the Python package (`discharge_inception/`), the SLURM shell wrapper (`Util/GenericArrayJob.sh`), example jobscripts, the `Exec/Rod` case, and this documentation source. No submodules are used, so a plain `git clone` is sufficient.

```
git clone https://github.com/SINTEF-Power-system-asset-management/discharge-inception.git
cd discharge-inception
```

1.3 Installation instructions

1.3.1 Create and activate a virtual environment

A virtual environment isolates the project's Python dependencies from the system Python and from other projects on the same machine. This is especially important on HPC clusters, where the system Python may be managed by administrators and `pip install` into it is either forbidden or inadvisable.

The virtual environment directory is conventionally named `.venv` and placed at the repository root. Because compute nodes need to activate the same environment, the `.venv` directory must live on a shared filesystem that is accessible from all nodes — typically your home directory or a project scratch space, both of which are usually network-mounted on HPC clusters. Do **not** place `.venv` on a node-local scratch directory (e.g. `/tmp`), as it will not be visible to other nodes.

From the repository root:

```
python -m venv .venv
source .venv/bin/activate        # Linux / macOS
# .venv\Scripts\activate         # Windows
```

1.3.2 Install the package

Install in *editable* mode so that local edits to the `discharge_inception/` source directory take effect immediately without reinstalling. Editable mode also means that the `discharge-inception` command-line entry point is registered in the virtual environment and will always run the current state of the checked-out source.

The `[plot]` optional dependency group adds `matplotlib` and `scipy`, which are used by the post-processing scripts in `PostProcess/` but are not required to run the configurator or submit SLURM jobs:

```
pip install -e .                 # core only
pip install -e ".[plot]"         # with matplotlib + scipy
```

1.3.3 Verify

After installing the package, confirm that the `discharge-inception` entry point is registered in the active virtual environment and that the package imports correctly. The `which` command should resolve to the `.venv/bin/` directory, confirming that the system Python (or another environment) is not being used by mistake. Running `--help` exercises the full import chain — including `configurator` and `config_util` — so any missing dependency or broken import will surface here rather than at job-submission time.

```
which discharge-inception              # should point into .venv/bin/
discharge-inception --help
```

1.3.4 Environment variables

Two environment variables configure runtime behaviour on compute nodes. Both paths must be reachable from all compute nodes (typically a shared filesystem such as `$HOME` or a project scratch space). Because `GenericArrayJob.sh` runs on whatever node SLURM allocates, it cannot rely on paths that were valid only on the login node — using absolute paths on a shared filesystem avoids node-local ambiguity entirely.

DISCHARGE_INCEPTION_VENV
: Absolute path to the `.venv` directory created during installation. `GenericArrayJob.sh` sources `$DISCHARGE_INCEPTION_VENV/bin/activate` on each compute node before invoking the Python jobscript, ensuring the correct interpreter and installed packages are used. If this variable is unset or empty, the script falls back to whatever `python` is on the default `PATH`, which may not have `discharge_inception` installed.

DISCHARGE_INCEPTION_SLURM_CONFIG
: Absolute path to the `slurm.toml` configuration file. The configurator (`discharge-inception run`) reads this file at submission time to build `sbatch` resource arguments, and `GenericArrayJob.sh` reads it on compute nodes to load the required cluster modules. Job scripts also call `load_slurm_config()` at runtime to retrieve the MPI launcher name and per-stage resource limits. See *SLURM configuration* for the full file format and all supported keys.

Add both exports to your `.bashrc`, SLURM prologue, or cluster environment module so they are present on both login and compute nodes:

```
export DISCHARGE_INCEPTION_VENV=/path/to/repo/.venv
export DISCHARGE_INCEPTION_SLURM_CONFIG=/path/to/repo/slurm.toml
```

Note: As an alternative to `DISCHARGE_INCEPTION_SLURM_CONFIG`, standard SLURM environment variables (`SBATCH_ACCOUNT`, `SBATCH_TIMELIMIT`, etc.) are respected by `sbatch` and can be used to supply resource requests without a `slurm.toml` file.

1.4 SLURM configuration

Resource requests are stored in `slurm.toml` (path given by `DISCHARGE_INCEPTION_SLURM_CONFIG`). This keeps all cluster-specific settings in one place and out of shell scripts.

Listing 1: slurm.toml

```
[slurm]
account        = ""        # SLURM account; empty uses cluster default
```

(continues on next page)

(continued from previous page)

```
partition      = ""        # SLURM partition; empty uses cluster default
mpi            = "mpirun"  # MPI launcher: "mpirun", "srun", or "mpiexec"
modules        = []        # Modules to load, e.g. ["foss/2023a", "HDF5/1.14.0-gompi-
↪2023a"]
nodes          = 1         # Default number of nodes
tasks_per_node = 16        # Default MPI tasks per node

[slurm.inception]
# Per-stage overrides for inception (database) runs
tasks_per_node = 4
time = "0-00:30:00"

[slurm.plasma]
# Per-stage overrides for plasma (voltage) runs
tasks_per_node = 16
time = "0-02:00:00"
```

`GenericArrayJob.sh` reads the `modules` list from `slurm.toml` and calls `module load` for each entry. Job scripts call `build_sbatch_resource_args()` to translate the `[slurm.<stage>]` section into `sbatch` command-line flags.

The `DISCHARGE_INCEPTION_SLURM_CONFIG` variable must be set (and exported) before submitting any job so that compute nodes can find the file.

As an alternative to `slurm.toml`, standard SLURM environment variables (`SBATCH_ACCOUNT`, `SBATCH_TIMELIMIT`, etc.) are honoured by `sbatch` directly and do not require a config file.

1.5 The CLI

1.5.1 Overview

All functionality is accessed through the `discharge-inception` command. Run the following to see every available subcommand:

```
$ discharge-inception --help
```

```
usage: discharge-inception [-h] command ...

Parametric study configurator for chombo-discharge simulations.

positional arguments:
  command
    run                   Configure and submit a parametric study.
    ls                    List runs and parameter settings in a study directory.
    analyze-time-series   Extract, smooth, differentiate, and filter time-series
                          data from a plasma log.
    extract-inception-voltages
                          Extract inception voltages from a pdiv_database and
                          write NetCDF/CSV.
    gather-plasma-event-logs
                          Gather plasma event logs from a database and write a
                          CSV summary.
```

(continues on next page)

(continued from previous page)

```
    plot-delta-e-rel      Batch-plot Delta E(rel) vs time for every run in a plasma
                          database.

options:
  -h, --help            show this help message and exit
```

Each subcommand accepts its own `--help` flag, e.g.:

```
discharge-inception extract-inception-voltages --help
```

1.5.2 discharge-inception run

Sets up directory structure and submits the initial SLURM array jobs.

```
usage: discharge-inception run [-h] [--output-dir OUTPUT_DIR] [--dim DIM]
                        [--verbose] [--logfile LOGFILE]
                        run_definition

positional arguments:
  run_definition        Parameter space definition (.json or .py with top_object).

options:
  -h, --help            show this help message and exit
  --output-dir OUTPUT_DIR
                        Output directory for study result files. (default: study_results)
  --dim DIM             Dimensionality of simulations. Must match chombo-discharge
                        compilation. (default: 3)
  --verbose             Increase verbosity.
  --logfile LOGFILE     Log file; rotated automatically each invocation.
                        (default: configurator.log)
```

1.5.3 discharge-inception ls

Prints a table of runs, parameter values, and completion status (`[done]` if `report.txt` is present).

```
usage: discharge-inception ls [-h] study_dir [study_dir ...]

positional arguments:
  study_dir   Study output directory containing index.json (e.g. pdiv_database/).
```

Example output:

```
~/my_rod_study/PDIV_DB  (2 runs)
  run  pressure  geometry_radius  K_max
  ---  --------  ---------------  -----
  run_0  100000  0.001            12  [done]
  run_1  200000  0.001            12
```

1.5.4 discharge-inception analyze-time-series

Extracts, optionally smooths, and differentiates time-series data from a plasma simulation log. See *Analysing simulation time-series output* for full documentation of options and output columns.

```
usage: discharge-inception analyze-time-series [-h] [-i INPUT] [-o OUTPUT]
                                               [--sg] [--sg-window SG_WINDOW]
                                               [--sg-order SG_ORDER] [--lp]
                                               [--lp-tau LP_TAU]
```

Example:

```
discharge-inception analyze-time-series -i run_0/pout.0 -o run_0/pout.out --sg
```

1.5.5 discharge-inception extract-inception-voltages

Extracts the six inception voltages from every run in a `pdiv_database` directory and writes a NetCDF or CSV dataset. See *Extracting partial discharge inception voltages* for full documentation.

```
usage: discharge-inception extract-inception-voltages [-h] [--output OUTPUT]
                                                      [--format {netcdf,csv}]
                                                      [--plot PARAM [PARAM ...]]
                                                      [--select KEY=VALUE ...]
                                                      [--voltage {min,streamer,townsend,
↪all}]
                                                      db_dir
```

Example:

```
discharge-inception extract-inception-voltages study_results/pdiv_database
```

1.5.6 discharge-inception gather-plasma-event-logs

Scans the tail of each run's `pout.0` log, classifies every run as completed, inception, convergence failure, or abort, and writes a CSV summary. See *Identifying discharge inception events from plasma logs* for full documentation.

```
usage: discharge-inception gather-plasma-event-logs [-h] [--output PATH]
                                                    [--tail N] [--plot PARAM]
                                                    [--no-output]
                                                    db_dir
```

Example:

```
discharge-inception gather-plasma-event-logs study_results/plasma_database
```

1.5.7 discharge-inception plot-delta-e-rel

Batch-plots Delta E(rel) vs time for every run in a plasma database, saving one PNG per run. See *Plotting relative electric field change over time* for full documentation.

```
usage: discharge-inception plot-delta-e-rel [-h] [--prefix PREFIX]
                                            [--output-dir DIR]
                                            db_dir
```

Example:

```
discharge-inception plot-delta-e-rel study_results/plasma_database --output-dir plots/
```

CHAPTER

TWO

ARCHITECTURE

2.1 Overview

`discharge-inception` is a framework for submitting, tracking, and post-processing parametric chombo-discharge studies on SLURM clusters. A study is declared as a Python `Runs.py` file and submitted via the `discharge-inception` CLI. The CLI creates run directories, injects parameters, and hands off to SLURM. Everything that happens *inside* a SLURM job is driven by one of the three Python jobscripts.

The framework is built around a **two-stage pipeline** concept:

- A *database* phase (fast / lightweight, e.g. a discharge-inception sweep) runs first and produces intermediate data such as voltage tables.
- A *study* phase (full plasma simulation) depends on the database completing (via SLURM `--dependency=afterok`) and uses the database results to configure detailed runs.

2.2 Repository layout

Path	Role
`discharge_inception/`	Installable Python package: `configurator`, `config_util`, `cli`
`Util/GenericArrayJob.sh`	Portable SLURM bash wrapper; the only `#SBATCH` script
`GenericArrayJobJobscript.py`	Leaf-level voltage solver runner (called via `jobscript_symlink`)
`Exec/Rod/Studies/DischargeInceptionJobscript.py`	Inception database jobscript — runs the inception solver, validates range
`Exec/Rod/Studies/PlasmaJobscript.py`	Study orchestrator — reads inception results, creates voltage subdirs, submits child array
`slurm.toml`	Cluster resource configuration (MPI launcher, modules, per-stage overrides)
`Exec/Rod/`	Flat layout: source, headers, and data files for the Rod case
`Exec/Rod/Studies/PressureStudy/Runs.py`	Example parameter space definition
`PostProcess/`	Summary and plotting scripts

2.3 Databases and studies

The top-level `Runs.py` file defines a `top_object` dictionary containing two lists: `databases` and `studies`:

```
top_object = dict(
    databases=[inception_stepper],
    studies=[plasma_study_1]
)
```

A **database** is a lightweight first step that generates intermediate data. A **study** depends on one or more databases via SLURM `afterok` ordering; the configurator submits study jobs with a dependency on the database job ID.

The parameter space of a study is the Cartesian product of its own parameters, filtered to only the combinations that match a completed database run.

Example: a database sweeping 5 pressures produces 5 SLURM array tasks. A study sweeping those same 5 pressures x 3 rod radii produces 15 tasks, all chained after the database completes.

2.4 Run definition

Each database or study entry is a dictionary with these configurable fields:

`identifier`
: Unique string name for this database/study. Used as the symlink name in dependent study directories (e.g. `study0/inception_stepper -> ../PDIV_DB`).

`output_directory`
: Sub-directory inside `--output-dir` where this stage's files are written.

`output_dir_prefix`
: Prefix for individual run directories (default: `"run_"`).

`program`
: Executable to run. The token `{DIMENSIONALITY}` is replaced at setup time with the value of `--dim`. The binary is copied/symlinked into the output directory hierarchy.

`job_script`
: Python script that drives the actual SLURM work for this stage. The configurator creates a symlink `jobscript_symlink -> <job_script>` in the stage directory so `GenericArrayJob.sh` can call it generically.

`job_script_dependencies`
: List of files the jobscript itself needs at runtime (e.g. `GenericArrayJob.sh`, `ParseReport.py`). These are copied into the stage's top-level directory.

`required_files`
: List of files copied into **every** per-run `run_N/` directory — typically `*.inputs`, `chemistry.json`, and other data files needed by the executable.

`parameter_space`
: Dictionary of named parameters. See *Defining parameter spaces*.

The key distinction between `job_script_dependencies` and `required_files`:

- `job_script_dependencies` — files the jobscript at the stage level needs (present once in the stage directory).
- `required_files` — files needed by every invocation of the executable (copied into every `run_N/` directory).

A realistic example:

```
inception_stepper = {
    'identifier': 'inception_stepper',
    'output_directory': 'PDIV_DB',
    'program': rod_dir + 'main{DIMENSIONALITY}d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex',
    'job_script': 'DischargeInceptionJobscript.py',
    'job_script_dependencies': [
        'GenericArrayJob.sh',
        'ParseReport.py',
    ],
    'required_files': [
        'master.inputs',
        'chemistry.json',
        'electron_transport_data.dat',
        'detachment_rate.dat',
    ],
    'parameter_space': { ... }
}
```

Note the use of `{DIMENSIONALITY}` in the `program` field — this token is substituted with the value supplied via `--dim` on the command line.

2.5 Defining parameter spaces

A `parameter_space` is a dictionary whose keys are parameter names. Each value is another dictionary with these fields:

`database`
: (optional) Identifier of the database this parameter is linked to. Parameters that carry `"database"` create a SLURM `afterok` dependency between the study and the named database, and they restrict the Cartesian product to only the combinations that exist in the database.

`target`
: Filename (relative to the run directory) of the file to modify. Can be a `*.inputs` ParmParse file or a `*.json` file.

`uri`
: Address of the value to change within the target file. For `*.inputs` files this is a dot-separated ParmParse key string. For `*.json` files this is a list of nested dictionary keys; see *JSON URI syntax* for special syntax.

`values`
: List of values. Each element becomes one point in the parameter space. A 2nd-order list (list of lists) drives *multiple simultaneous writes* to parallel targets.

The Cartesian product of all `values` lists determines the number of runs. Database-linked parameters restrict that product to matching combinations.

```
'parameter_space': {
    "pressure": {
        "target": "chemistry.json",
        "uri": ["gas", "law", "ideal_gas", "pressure"],
        "values": [1e5, 2e5, 3e5]        # factor 3
    },
    "geometry_radius": {
        "target": "master.inputs",
```

(continues on next page)

(continued from previous page)

```
        "uri": "Rod.radius",
        "values": [1e-3, 2e-3]           # factor 2
    }
}
```

The above yields 6 run directories (3 x 2).

2.6 JSON URI syntax

When a parameter's `target` is a `.json` file, the `uri` field is a list of keys that traverses the nested JSON hierarchy. Two special notations allow searching inside JSON *lists* (arrays of objects):

`+["field"="value"]`
: Find the object in the list whose `"field"` equals `"value"`. The object **must** exist; raises an error if not found.

`*["field"="value"]`
: Find the object in the list whose `"field"` equals `"value"`, **or create it** if absent.

`<chem_react>`
: Hint to the parser that `value` is a chombo-discharge chemical reaction string; comparison is semantic (ignores whitespace and ordering differences). See "Specifying reactions" in the chombo-discharge documentation.

Multiple parallel targets — when the second element of the uri list is itself a list, the same traversal step applies to *all* entries in that inner list simultaneously. This is used to write two fields at the same time:

```
"uri": [
    "photoionization",
    [
        '+["reaction"=<chem_react>"Y + (O2) -> e + O2+"]',
        '*["reaction"=<chem_react>"Y + (O2) -> (null)"]'
    ],
    "efficiency"
],
"values": [[1.0, 0.0]]
```

Example 1 — simple object search in a list:

```
{
    "parent": {
        "list-1": [
            {"field-name-0": "value_0"},
            {"field-name-1": {"target-field": "change-me!"}},
            {"field-name-2": "value_2"}
        ]
    }
}
```

```
"uri" = [
    "parent",
    "list-1",
    '+["field-name-1"]',   # finds the container object
    "field-name-1",        # selects the child object
```

(continues on next page)

(continued from previous page)

```
    "target-field"         # the actual target
]
```

Example 2 — searching by a specific value:

```
{
    "parent": {
        "list-level-1": [
            {"field-name-0": "value_0"},
            {"field-name-1": "value_1_0", "target-field": "dont-change-me!"},
            {"field-name-1": "value_1_1", "target-field": "change-me!"},
            {"field-name-2": "value_2"}
        ]
    }
}
```

```
"uri" = [
    "parent",
    "list-level-1",
    '+["field-name-1"="value_1_1"]',   # finds the correct object
    "target-field"                      # the actual target
]
```

Example 3 — traversing two list levels:

```
{
    "parent": {
        "list-level-1": [
            {"field-name-1": "value_1_1",
             "target-field": [
                 {"search-field": "some-value", "target2-field": "dont-change!"},
                 {"search-field": "search-value", "target2-field": "change-me!"}
             ]}
        ]
    }
}
```

```
"uri" = [
    "parent",
    "list-level-1",
    '+["field-name-1"="value_1_1"]',
    "target-field",
    '+["search-field"="search-value"]',
    "target2-field"
]
```

Note: If searching for an object in a list where the search key is itself a JSON object, the value part can be omitted: `+["field-name"]`.

2.7 Dummy parameters

A *dummy* parameter has no `target` or `uri` — only a `name` and a `values` list (and optionally `database`). It passes configuration options to jobscripts through `parameters.json` without modifying any simulation input files.

A dummy parameter with a **single value** does not expand the parameter space (contributes a factor of 1 to run count):

```
main_study = {
    ...
    "parameter_space": {
        "K_min": {
            "values": [6.0]    # single value - does not add runs
        },
    }
}
```

The value is still written to `index.json`, `structure.json`, and `parameters.json`, making it available to jobscripts at runtime.

2.8 Output directory structure

After `discharge-inception run` completes (and while the SLURM jobs are running), the output tree looks like:

```
$ ls -R --file-type ~/my_rod_study
.:
PDIV_DB/  study0/

./PDIV_DB:
array_job_id  DischargeInceptionJobscript.py  GenericArrayJob.sh
index.json    master.inputs                   ParseReport.py
main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex
run_0/  structure.json  chemistry.json  electron_transport_data.dat  detachment_rate.dat
jobscript_symlink@

./PDIV_DB/run_0:
chk/  master.inputs  parameters.json  plt/  pout.*  main@
chemistry.json  electron_transport_data.dat  detachment_rate.dat

./study0:
array_job_id  chemistry.json  GenericArrayJob.sh  inception_stepper@
index.json    master.inputs   PlasmaJobscript.py
main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex
run_0/  structure.json  jobscript_symlink@

./study0/run_0:
chemistry.json  detachment_rate.dat  electron_transport_data.dat
master.inputs   parameters.json      main@
```

Key files and symlinks:

`array_job_id`
: Contains the SLURM array job ID for this stage as a single integer.

index.json
Maps integer run indices to parameter tuples. Format:

```
{"prefix": "run_", "keys": ["pressure", "geometry_radius"], "index": {"0": [1e5, 1e-
↪3], ...}}
```

structure.json
Parsed export of the run definition for this stage. Jobscripts read it to get metadata such as the run prefix and `required_files` list.

run_N/parameters.json
Named parameter dict for run N. Convenient for inspection and for jobscripts that need to locate a matching database run.

jobscript_symlink@
Points to the jobscript for this stage (e.g. `DischargeInceptionJobscript.py`). `GenericArrayJob.sh` calls `python ./jobscript_symlink` without knowing which script it is.

run_N/main@
Symlink pointing to the executable in the parent stage directory.

study0/inception_stepper@
Symlink pointing to `../PDIV_DB` — gives the plasma study jobscript direct access to database results.

2.9 The full call chain

```
discharge-inception run <Runs.py>          [CLI -- discharge_inception/configurator.py]
  |  Creates run dirs, writes index.json / parameters.json per run,
  |  symlinks jobscript_symlink, writes DISCHARGE_INCEPTION_SLURM_CONFIG,
  |  then submits:
  |
  \- sbatch --array=0-N GenericArrayJob.sh        [SLURM entry-point]
       |  Loads cluster modules from slurm.toml, activates venv, then runs:
       |
       \- python ./jobscript_symlink               [jobscript dispatch]
            |
            +--- DischargeInceptionJobscript.py    [inception database runs]
            |      Navigates to run_<id>/ via index.json, runs the inception
            |      solver, validates max_voltage, optionally reruns.
            |      Output: report.txt in each run directory.
            |
            \--- PlasmaJobscript.py                [plasma study runs]
                   Navigates to run_<id>/ via structure.json prefix, looks up
                   the matching inception database run, reads its report.txt,
                   builds a voltage table, creates voltage_<i>/ subdirs,
                   then submits a SECOND sbatch array:
                   |
                   \- sbatch --array=0-M GenericArrayJob.sh   [voltage array]
                        \- python ./jobscript_symlink
                             \--- GenericArrayJobJobscript.py  [voltage runs]
                                    Navigates to voltage_<id>/ via index.json,
                                    runs the plasma solver for one voltage.
```

CLI (``configurator.py``) — Reads the `Runs.py` definition, expands the Cartesian parameter space, creates the

full directory tree, copies executables and data files, writes `index.json` / `parameters.json` / `structure.json` per stage, creates `jobscript_symlink`, and submits the initial `sbatch` arrays. Study arrays are submitted with `--dependency=afterok:<db_job_id>` to enforce ordering.

``**GenericArrayJob.sh**`` — The only `#SBATCH` script in the project. It is completely resource-agnostic; all resource values are injected at submission time by the Python jobscripts via `build_sbatch_resource_args()`. It reads `DISCHARGE_INCEPTION_SLURM_CONFIG` to load cluster modules and activate the virtual environment, then calls `python ./jobscript_symlink`.

``**DischargeInceptionJobscript.py**`` — Reads `index.json` to find its run directory, runs the inception solver, parses `report.txt` to check the voltage range, and reruns with an updated voltage ceiling if necessary.

``**PlasmaJobscript.py**`` — Reads `structure.json` to find its run directory, locates the matching database run, extracts a filtered voltage table from the database `report.txt`, creates per-voltage subdirectories with injected parameters, and submits a child SLURM array for the voltage sweep.

``**GenericArrayJobJobscript.py**`` — Leaf-level runner. Reads `index.json` in the voltage subdirectory, navigates to `voltage_<id>/`, and launches the plasma solver via MPI for a single voltage point.

2.10 Script roles

Script	Role	Reads	Called by
`Util/GenericArrayJob.sh`	SLURM wrapper; loads modules, activates venv	`DISCHARGE_INCEPTION_SLURM_CONFIG` -> `slurm.toml`	`sbatch --array=...`
`GenericArrayJobJobscript.py`	Runs plasma solver for one voltage	`index.json`, `*.inputs`, `slurm.toml`	Second `sbatch` in `PlasmaJobscript.py`
`DischargeInceptionJobscript.py`	Runs inception solver, validates voltage range	`index.json`, `*.inputs`, `report.txt`	First `sbatch` on database study
`PlasmaJobscript.py`	Orchestrates voltage sweep: reads inception results, creates subdirs, submits child array	`structure.json`, `parameters.json`, `../inception_stepper/`, `report.txt`	First `sbatch` on plasma study
`discharge_inception/configurator.py`	Expands parameter space, creates dirs, submits initial arrays	`Runs.py` / `top_object`	`discharge-inception run` CLI
`discharge_inception/config_util.py`	URI injection, file helpers, SLURM task ID, jobscript setup	Called by all jobscripts	All jobscripts

CHAPTER

THREE

JOBSCRIPTS GUIDE

3.1 Setting up a simulation directory

`Setup/setup.py` scaffolds a new simulation directory under `Exec/` (or a user-specified parent). The generated directory contains `main.cpp`, `GNUmakefile`, `template.inputs`, and any physics-model dependency files (e.g. `chemistry.json`).

The produced code follows the **dual-mode** pattern: a single binary that runs either the `DischargeInceptionStepper` (inception mode) or `ItoKMCStepper` (plasma mode), selected at runtime via `app.mode` in the inputs file.

3.1.1 How it works

`setup.py` delegates to three helper modules:

`app_main.py`
: Writes `main.cpp` with parametrized `#include` directives, solver type aliases (`I`, `C`, `R`, `F`), and a dual-mode `main()` that branches on `app.mode`.

`app_options.py`
: Collects all `.options` files for the selected components and concatenates them into `template.inputs`, prefixed by the `app.mode` and `plasma.voltage` runtime controls.

`app_inc.py`
: Reads `CD_{physics}.inc` from the plasma-model directory and copies any listed dependency files (e.g. `chemistry.json`) into the new application directory.

3.1.2 Arguments

Argument	Type	Default	Description
-discharge_home	str	$DISCHARGE_HOME	Path to the chombo-discharge source tree. Required; the script exits with an error if this is empty and the environment variable is not set.
-base_dir	str	Exec/ (project root)	Parent directory under which the new application subdirectory is created.
-app_name	str	MyApplication	Name of the new application subdirectory.
-geometry	str	RegularGeometry	Chombo-Discharge computational geometry class.
-physics	str	ItoKMCJSON	ItoKMC plasma physics model class.
-ito_solver	str	ItoSolver	Ito-diffusion solver type (I template parameter).
-cdr_solver	str	CdrCTU	Convection-diffusion-reaction solver type (C template parameter).
-rte_solver	str	McPhoto	Radiative-transfer solver type (R template parameter).
-field_solver	str	FieldSolverGMG	Poisson/field solver type (F template parameter).
-plasma_stepper	str	ItoKMCBackgroundEvaluator	ItoKMC time-stepper used in plasma mode.
-plasma_tagger	str	ItoKMCStreamerTagger	Cell tagger used in plasma mode. Pass none to disable tagging.

3.1.3 Example

The following invocation reproduces the Exec/Rod configuration. Run it from the project root:

Listing 1: Scaffold a Rod application

```
python3 Setup/setup.py \
    -discharge_home "$DISCHARGE_HOME" \
    -base_dir Exec \
    -app_name MyRod \
    -geometry Rod \
    -physics ItoKMCJSON \
    -plasma_stepper ItoKMCBackgroundEvaluator \
    -plasma_tagger ItoKMCStreamerTagger
```

This creates Exec/MyRod/ containing:

- main.cpp — dual-mode driver configured for the Rod geometry.
- GNUmakefile — ready-to-use build file.
- template.inputs — complete options for every selected component, with app.mode and plasma.voltage at the top.
- chemistry.json — dependency copied from the ItoKMCJSON model directory via CD_ItoKMCJSON.inc.

3.2 Where jobscripts fit

Rather than requiring users to author their own jobscripts, the framework ships two ready-made scripts in `Scripts/`:

`Scripts/DischargeInceptionJobscript.py`
: Runs the **database stage** (inception stepper). Each SLURM task processes one parameter combination: it invokes `DischargeInceptionStepper`, checks the result against `DischargeInceptionTagger.max_voltage`, and auto-corrects and reruns if the stepper exceeded the tagger limit. See *Specifying the inception database options*.

`Scripts/PlasmaJobscript.py`
: Runs the **plasma stage** (ItoKMC). It reads the database results, selects voltages from the K-range, creates per-voltage subdirectories, and submits the voltage SLURM array. See *Specifying the plasma simulation options*.

Both scripts are invoked through `GenericArrayJob.sh`, which is the SLURM entry point: it activates the environment and calls `python ./jobscript_symlink`, which resolves to the appropriate script for the stage. See *GenericArrayJob.sh — the SLURM wrapper* for details on the SLURM wrapper itself, and *The full call chain* for a full picture of how all the pieces connect.

3.3 GenericArrayJob.sh — the SLURM wrapper

`Util/GenericArrayJob.sh` is the **only** `#SBATCH` script in the project. It contains no hardcoded resource requests (nodes, tasks, time, account). Instead, resource values are injected at submission time by the Python jobscripts via `build_sbatch_resource_args()`, which reads `slurm.toml` (see *SLURM configuration*).

The only `#SBATCH` directives it does contain redirect stdout/stderr to per-array-task log files:

Listing 2: Util/GenericArrayJob.sh

```
#!/bin/bash
# Generic SLURM array job launcher for discharge-inception.
#
# Resource requests (account, partition, ntasks, time) are intentionally
# absent here so the script is portable across clusters. Supply them via:
#   - sbatch CLI arguments:  sbatch --account=X --ntasks=N --time=HH:MM:SS ...
#   - SLURM environment variables: export SBATCH_ACCOUNT=X before submitting
#   - The Python job scripts read slurm.toml and pass these automatically
#     when they invoke sbatch themselves (e.g. for the voltage sub-array).

#SBATCH --output=R-%x.%A-%a.out
#SBATCH --error=R-%x.%A-%a.err

set -o errexit
set -o nounset

# Load cluster modules listed in slurm.toml (requires DISCHARGE_INCEPTION_SLURM_CONFIG
# to be set and exported before submitting the job). Uses the system python3
# (before venv activation) to parse the TOML. The block is skipped entirely on
# systems without the 'module' command or without the config file.
if command -v module > /dev/null 2>&1 \
        && [ -n "${DISCHARGE_INCEPTION_SLURM_CONFIG:-}" ] \
        && [ -f "${DISCHARGE_INCEPTION_SLURM_CONFIG}" ]; then
    while IFS= read -r mod; do
```

(continues on next page)

(continued from previous page)

```
        [ -n "$mod" ] && module load "$mod"
    done < <(python3 -c "
import sys, tomllib
with open(sys.argv[1], 'rb') as f:
    c = tomllib.load(f)
for m in c.get('slurm', {}).get('modules', []):
    print(m)
" "${DISCHARGE_INCEPTION_SLURM_CONFIG}")
fi

if [ -n "${DISCHARGE_INCEPTION_VENV:-}" ]; then
    source "$DISCHARGE_INCEPTION_VENV/bin/activate"
fi

python ./jobscript_symlink
exit $?
```

`GenericArrayJob.sh` must be listed as a `job_script_dependency` in every run definition entry so the configurator copies it into the stage directory (see *Run definition*).

3.4 Specifying the inception database options

`Scripts/DischargeInceptionJobscript.py` is the jobscript for the **database stage**. Each SLURM task executes one parameter combination:

1. Runs `DischargeInceptionStepper` to compute the ionisation integral over a voltage sweep.
2. Validates the result against `DischargeInceptionTagger.max_voltage`.
3. If the stepper exceeded the tagger limit, auto-corrects the voltage range and reruns until a valid result is obtained.

The script does not need to be edited directly. All configuration is driven by the run definition — specifically the `parameter_space` and `input_overrides` fields — which the configurator writes into the `.inputs` file before the job runs.

3.4.1 Parameter space

Keys listed under `parameter_space` in the run definition become the free parameters of the database sweep. The configurator injects one value per key into the `.inputs` file for each SLURM task, so the stepper sees the correct combination.

See *Defining parameter spaces* and *Run definition* for the full syntax.

3.4.2 Input overrides

Keys listed under `input_overrides` are fixed values injected into every `.inputs` file regardless of the task index. Use this to set stepper or tagger settings that are constant across the sweep (e.g. `DischargeInceptionStepper.max_steps`).

See *Run definition* for details.

3.4.3 job_script_options

This script has no user-configurable runtime knobs beyond what is already expressed in the `.inputs` file. The `job_script_options` field of the run definition is not read by `DischargeInceptionJobscript.py` and may be omitted.

3.5 Specifying the plasma simulation options

`Scripts/PlasmaJobscript.py` is the jobscript for the **plasma stage**. For each parameter combination it:

1. Reads the database results produced by the inception stage.
2. Selects voltages that fall within the configured K-range.
3. Creates a per-voltage subdirectory for each selected voltage.
4. Submits the voltage SLURM array (one task per voltage).

The script does not need to be edited directly. Runtime behaviour is controlled through the `job_script_options` block in the run definition.

3.5.1 job_script_options reference

All keys are read from `parameters.json` at runtime.

Key	Type	Default	Description
`plasma_polarity`	`str`	both	`'positive'` selects K(+) voltages only; `'negative'` selects K(-) voltages only. Omit the key (or set it to `'both'`) to include both polarities.
`K_min`	`float`	`0`	Lower bound on the ionisation integral. Database rows with K below this value are excluded.
`K_max`	`float`	from `.inputs`	Upper bound on the ionisation integral. If not set, the value of `DischargeInceptionStepper.limit_max_K` from the `.inputs` file is used as the fallback.
`N_voltages`	`int`	all rows	If set, interpolates *N* evenly-spaced voltage points per polarity instead of using the raw report rows from the database.
`particle_mode`	`str`	`'single'`	`'single'` — launches one seed electron per task. `'sphere'` — distributes seed electrons over a sphere.
`num_particles`	`int`	`1`	Particle weight in `'single'` mode, or total particle count in `'sphere'` mode.
`sphere_radius`	`float`	—	Sphere radius in metres. **Required** when `particle_mode='sphere'`.
`sphere_center`	`list[float]`	interpolated	Explicit sphere centre as `[x, y, z]`. Defaults to the K-weighted position read from the database.

3.5.2 Parameter space and input overrides

The plasma stage supports the same `parameter_space` and `input_overrides` syntax as the database stage. See *Run definition* for details.

3.5.3 Example

The following snippet shows a `job_script_options` block inside a run definition:

```
{
  "run_name": "rod_plasma",
  "jobscript": "Scripts/PlasmaJobscript.py",
  "parameter_space": {
    "gas_pressure": [1e5, 2e5, 4e5]
  },
  "job_script_options": {
    "plasma_polarity": "positive",
    "K_min": 0.5,
    "K_max": 5.0,
    "N_voltages": 10,
    "particle_mode": "sphere",
    "num_particles": 500,
    "sphere_radius": 1e-4
  }
}
```

CHAPTER

FOUR

POST-PROCESSING

4.1 Extracting partial discharge inception voltages

4.1.1 Overview

`ExtractInceptionVoltages.py` scans a `pdiv_database` directory and collects the six inception voltages written by DischargeInceptionStepper into each run's `report.txt` file. It assembles these values into a structured dataset indexed by the sweep parameters and writes either a NetCDF file (default) or a CSV file. A summary table is always printed to standard output.

The six voltage quantities extracted are:

- **Minimum inception voltage** — positive and negative polarity
- **Streamer inception voltage** — positive and negative polarity
- **Townsend inception voltage** — positive and negative polarity

Values that the solver could not compute (stored internally as `DBL_MAX`) are replaced with `NaN` in the output.

The script reads `index.json` (required) and `structure.json` (optional, used to preserve the intended coordinate ordering) from the database root directory.

Note: Optional dependencies

- `xarray` — required for the default NetCDF output format and for all plotting functionality. If it is not installed, use `--format csv` to produce a plain CSV file instead. Install with:

```
pip install xarray netcdf4
```

- `matplotlib` — required only when `--plot` is used. If it is absent the script exits with an error when plotting is requested. Install with:

```
pip install matplotlib
```

Neither package is needed for basic CSV extraction.

4.1.2 Options

db_dir

Positional (required). Path to the `pdiv_database` directory. The directory must contain an `index.json` file.

--output `PATH`

Path for the output file. Defaults to `<db_dir>/inception_voltages.nc` (NetCDF) or `<db_dir>/inception_voltages.csv` (CSV).

--format `{netcdf,csv}`

Output format. `netcdf` (default) writes a multi-dimensional xarray/NetCDF dataset. `csv` writes a flat table with one row per run. Use `csv` when `xarray` is not available.

--plot `PARAM [PARAM]`

Generate an interactive Matplotlib figure after writing the output file. Supply one parameter name for a line plot (inception voltage vs. that parameter) or two parameter names for a pair of heat-maps. Parameters must match keys defined in `index.json`. Requires both `xarray` and `matplotlib`.

--select `KEY=VALUE [KEY=VALUE ...]`

When plotting, fix one or more sweep parameters at the given values so that a lower-dimensional slice can be displayed. For each dimension not listed in `--plot` and not covered by `--select`, the script defaults to the first available coordinate value and prints a warning. Example: `--select pressure=1e5 temperature=300`.

--voltage `{min,streamer,townsend,all}`

Filter which voltage type(s) appear in the plot. Defaults to `all`. Has no effect on the written output file.

4.1.3 Example

Extract all inception voltages from a completed database into a NetCDF file, then display a line plot of all three voltage types against the `gap_distance` parameter:

```
python PostProcess/ExtractInceptionVoltages.py \
    results/pdiv_database \
    --plot gap_distance
```

To run without `xarray` installed, write a CSV instead:

```
python PostProcess/ExtractInceptionVoltages.py \
    results/pdiv_database \
    --format csv \
    --output results/inception_voltages.csv
```

For a two-parameter sweep over `gap_distance` and `pressure`, produce heat-maps showing only the streamer inception voltage:

```
python PostProcess/ExtractInceptionVoltages.py \
    results/pdiv_database \
    --plot gap_distance pressure \
    --voltage streamer
```

4.2 Identifying discharge inception events from plasma logs

4.2.1 Overview

`GatherPlasmaEventLogs.py` scans a plasma simulation database directory and reads the tail of each run's `pout.0` log file to extract final simulation state and event information. It writes a structured CSV file and always prints a human-readable summary table to standard output.

The following information is extracted for every run:

- **final_step** — last reported time-step number
- **final_time** — last reported simulation time
- **final_dt** — last reported time-step size
- **inception** — whether a discharge inception event was detecte d
- **convergence_failures** — number of Poisson solver convergence failures
- **other_abort** — whether any other unexpected abort was found
- **status** — derived run outcome: `completed`, `inception`, `convergence_failure`, `abort`, or `not_found`

Status is assigned with the following priority (highest first): `not_found` -> `inception` -> `convergence_failure` -> `abort` -> `completed`.

The script reads `index.json` (required) and `structure.json` (optional, used to preserve the intended parameter ordering) from the database root directory.

Note: Optional dependency

`matplotlib` — required only when `--plot` is used. If it is absent the script exits with an error when plotting is requested. Install with:

```
pip install matplotlib
```

No optional packages are needed for basic CSV extraction.

4.2.2 Options

db_dir

Positional (required). Path to the plasma simulation database directory. The directory must contain an `index.json` file.

--output `PATH`

Path for the output CSV file. Defaults to `<db_dir>/plasma_event_log.csv`.

--tail `N`

Number of lines to read from the end of each `pout.0` file (default: `50`).

--plot `PARAM`

Generate an interactive Matplotlib scatter plot of final simulation time vs. *PARAM*, colour-coded by run status. *PARAM* must match a key defined in `index.json`. Requires `matplotlib`.

--no-output

Skip writing the CSV file. Only the summary table is printed to stdout.

4.2.3 Example

Gather event logs from a completed database and write the default CSV:

```
python PostProcess/GatherPlasmaEventLogs.py results/plasma_db
```

Read only the last 100 lines of each log and write to a custom path:

```
python PostProcess/GatherPlasmaEventLogs.py \
    results/plasma_db \
    --tail 100 \
    --output results/event_summary.csv
```

Print the summary table without writing any file, then show a scatter plot of final time vs the `voltage` parameter:

```
python PostProcess/GatherPlasmaEventLogs.py \
    results/plasma_db \
    --no-output \
    --plot voltage
```

4.3 Analysing simulation time-series output

4.3.1 Overview

`AnalyzeTimeSeries.py` reads a block-wise ASCII plasma simulation log (typically `pout.0`) and produces a clean, aligned 10-column `.dat` file together with an interactive Matplotlib figure.

For every `Driver::Time step report` block the script extracts eight physical fields:

Column	Name	Description
1	Time	Simulation time
2	dt	Time-step size
3	Delta E(max)	Maximum relative electric field change (%)
4	Delta E(rel)	Mean relative electric field change (%)
5	Q (ohmic)	Ohmic charge (C)
6	Q (electrode)	Electrode charge (C)
7	Sum (phi_optical)	Integrated optical ionisation source (dimensionless)
8	Sum (src_optical)	Integrated optical source rate (1/s)

Two further columns are derived by finite differentiation of the charge columns:

Column	Name	Description
9	I (ohmic)	Ohmic current dQ_{ohmic}/dt (A)
10	I (electrode)	Electrode current $dQ_{electrode}/dt$ (A)

The processing pipeline is:

1. Parse all time-step blocks from the log.
2. Optionally smooth columns 3-6 with a **Savitzky-Golay** filter (`--sg`).
3. Compute columns 9-10 as finite-difference derivatives of the (possibly smoothed) Q columns.

4. Optionally apply a **bidirectional exponential moving average** low-pass filter to columns 9-10 (`--lp`).
5. Write the aligned `.dat` file with a commented column header.
6. Display a 2 x 4 Matplotlib figure of columns 3–10 vs time.

The output file always begins with commented lines documenting the column layout, and all values are written in 8-digit scientific notation.

Note: Optional dependencies

- `scipy` — required when `--sg` is used. Install with:

```
pip install scipy
```

- `matplotlib` — required for the interactive plot. If absent, a warning is printed and the script continues without plotting. Install with:

```
pip install matplotlib
```

Neither package is needed to write the `.dat` file without smoothing.

4.3.2 Options

-i PATH, **--input** PATH

Path to the input ASCII log file. Defaults to `pout.0` in the current working directory.

-o PATH, **--output** PATH

Path for the output `.dat` file. Defaults to the input file's directory with the extension replaced by `.out` (e.g. `run_0/pout.out` when the input is `run_0/pout.0`).

--sg

Apply Savitzky-Golay smoothing to columns 3-6 (Delta E and Q columns) before computing the derivative columns. Requires `scipy`.

--sg-window N

Savitzky-Golay window length (default: `9`). Must be odd and greater than `--sg-order`; the value is automatically reduced if necessary to fit the data length.

--sg-order N

Savitzky-Golay polynomial order (default: `3`). Must be strictly less than `--sg-window`.

--lp

Apply a bidirectional exponential moving average low-pass filter to the derivative columns 9-10 after differentiation. Requires `--lp-tau`.

--lp-tau TAU

EMA time constant τ in seconds (default: none). Required when `--lp` is set. Smaller values pass more high-frequency content; larger values smooth more aggressively.

4.3.3 Example

Extract and write the default output without any filtering:

```
python PostProcess/AnalyzeTimeSeries.py -i run_0/pout.0 -o run_0/pout.out
```

Apply Savitzky-Golay smoothing (window 15, order 3) before differentiating:

```
python PostProcess/AnalyzeTimeSeries.py \
    -i run_0/pout.0 \
    -o run_0/pout.out \
    --sg --sg-window 15 --sg-order 3
```

Smooth the Q columns and then low-pass filter the resulting currents with a time constant of 1 ns:

```
python PostProcess/AnalyzeTimeSeries.py \
    -i run_0/pout.0 \
    --sg \
    --lp --lp-tau 1e-9
```

4.4 Plotting relative electric field change over time

4.4.1 Overview

`PlotDeltaERel.py` is a batch plotting tool that produces one PNG figure per run in a plasma simulation database. For each run it reads the corresponding `{prefix}{run_id}.0` log file, extracts the simulation time and the relative electric field change Δ E(rel) at every reported time step, and saves a labelled PNG file.

Note: This script targets an older database layout in which the log files sit directly inside the database root directory (e.g. `pout0.0`, `pout1.0`, …) rather than in per-run sub-directories. The file prefix can be changed with `--prefix`.

The figure for each run shows:

- **x-axis** — simulation time in nanoseconds
- **y-axis** — Δ E(rel) (%)
- **title** — the sweep-parameter key/value pairs for that run as read from `index.json`

Output PNGs are named `plt_0.png`, `plt_1.png`, … and written to *db_dir* by default (or to a custom directory via `--output-dir`).

Required dependencies: `numpy` and `matplotlib`.

4.4.2 Options

db_dir

Positional (required). Path to the plasma simulation database directory. The directory must contain an `index.json` file.

--prefix PREFIX

Prefix for the log filenames (default: `pout`). The script opens `<db_dir>/<PREFIX><run_id>.0` for each run.

--output-dir DIR

Directory in which output PNG files are saved (default: same as *db_dir*). The directory is created if it does not exist.

4.4.3 Example

Plot Δ E(rel) for all runs in a database, saving PNGs alongside the log files:

```
python PostProcess/PlotDeltaERel.py results/plasma_db
```

Use a non-default log prefix and write the figures to a dedicated plots folder:

```
python PostProcess/PlotDeltaERel.py \
    results/plasma_db \
    --prefix run \
    --output-dir results/plasma_db/plots
```

CHAPTER

FIVE

EXAMPLE — ROD CASE

5.1 Prerequisites

This chapter walks through the complete workflow for the `Exec/Rod` parametric study, from compilation through post-processing.

The Rod case demonstrates the **two-level database -> study pipeline**:

1. A discharge-inception database sweeps over pressures and rod radii to compute inception voltages.
2. A full plasma simulation study uses those inception voltages as its input, creating a sub-hierarchy of per-voltage runs.

Before starting, ensure the following are available:

- `discharge-inception` is installed — see *Installation instructions*.
- `DISCHARGE_HOME` is set and points to a compiled chombo-discharge installation — see *Prerequisites*.
- A SLURM scheduler is available on the machine — see *Prerequisites*.
- Both environment variables are exported — see *Environment variables*:

```
export DISCHARGE_INCEPTION_VENV=/path/to/repo/.venv
export DISCHARGE_INCEPTION_SLURM_CONFIG=/path/to/repo/slurm.toml
```

5.2 Compiling the executable

```
cd Exec/Rod
make -j4
```

This produces a single binary in `Exec/Rod/`:

```
main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex   (or main3d... for 3-D)
```

The binary handles **both** pipeline stages. The active mode is selected at runtime via `app.mode` in the `.inputs` file (`inception` or `plasma`). The `{N}` in the filename is replaced by the dimensionality supplied via `--dim`.

Note: Both pipeline stages share the same `chemistry.json` as their single source of truth for gas properties and transport data. In `inception` mode, α and η are computed via `ItoKMCJSON::computeAlpha/computeEta`, which derives them from the reaction network in `chemistry.json` — the same path used by `plasma` mode. No separate `transport_data.txt` is needed.

5.3 Running a smoke test

Verify the binary works before submitting any SLURM jobs. The default `master.inputs` sets `app.mode = inception`:

```
cd Exec/Rod
mpirun -n 4 ./main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex master.inputs
```

This runs a stationary inception-voltage sweep on the default rod geometry at 1 atm. Results are written to `report.txt`; plot files go to `plt/`.

To test plasma mode instead, override the mode (and optionally the voltage) on the command line — ParmParse arguments appended after the input filename take precedence over the file:

```
mpirun -n 4 ./main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex master.inputs \
    app.mode=plasma \
    plasma.voltage=40E3
```

Any `master.inputs` key can be overridden this way.

Note: The `plasma.voltage` key sets the constant applied voltage (in V) for the ItoKMC plasma simulation. The inception sweep does not use this value.

5.4 Inspecting the parameter space definition

Open `Exec/Rod/Studies/RadiusStudy.py` to inspect or adjust the parameter space. The top-level structure is:

```
top_object = dict(
    databases=[inception_stepper],
    studies=[plasma_study]
)
```

Both entries point to the flat `Exec/Rod/` directory:

```
rod_dir = '../'
```

Database (`inception_stepper`) — computes inception voltages. The free parameters are determined by the plasma study entries that carry `"database": "inception_stepper"` (see below). Fixed settings that apply to every database run are collected in `input_overrides`:

```
'input_overrides': {
    'mode': {
        'target': 'master.inputs',
        'uri': 'app.mode',
        'value': "inception"
    },
    "limit_max_K": {
        "target": "master.inputs",
        "uri": "DischargeInceptionStepper.limit_max_K",
        "value": 12
    },
```

(continues on next page)

(continued from previous page)

```
    "max_steps": {
        "target": "master.inputs",
        "uri": "Driver.max_steps",
        "value": 0
    },
    "plot_interval": {
        "target": "master.inputs",
        "uri": "Driver.plot_interval",
        "value": -1
    },
}
```

Study (`plasma_study`) — runs plasma simulations using the database results. `app.mode` is set to `plasma` via `input_overrides`. Parameters marked with `"database": "inception_stepper"` declare a SLURM dependency: study jobs will not start until the corresponding database job has completed, and the configurator ensures that each study run is paired with the matching database entry.

Plasma-only parameters (no `"database"` key) are swept independently:

```
'input_overrides': {
    'mode': {
        'target': 'master.inputs',
        'uri': 'app.mode',
        'value': "plasma"
    },
    "max_steps": {
        "target": "master.inputs",
        "uri": "Driver.max_steps",
        "value": 500
    },
},
'job_script_options': {
    'K_min': 6,
    'K_max': 12.0,
    'plasma_polarity': 'positive',
},
'parameter_space': {
    "geometry_radius": {
        "database": "inception_stepper",
        "target": "master.inputs",
        "uri": "Rod.radius",
        "values": [100E-6, 500E-6, 1e-3]
    },
    "pressure": {
        "database": "inception_stepper",
        "target": "chemistry.json",
        "uri": ["gas", "law", "ideal_gas", "pressure"],
        "values": [1e5]
    },
    "photoionization": {
        "target": "chemistry.json",
        "uri": [
            "photoionization",
```

(continues on next page)

(continued from previous page)

```
            [
                '+["reaction"=<chem_react>"Y + (O2) -> e + O2+"]',
                '*["reaction"=<chem_react>"Y + (O2) -> (null)"]'
            ],
            "efficiency"
        ],
        "values": [[1.0, 0.0]]
    },
}
```

The `photoionization` entry uses the `<chem_react>` match expression to locate reaction entries in `chemistry.json` by their reaction string. The `+` prefix requires the match to exist; `*` creates the entry if absent. A Python list at any level of the `uri` produces multiple parallel writes — here both reactions are written in a single parameter sweep step.

See *Defining parameter spaces*, *JSON URI syntax*, and *Databases and studies* for a full explanation of parameter space syntax, JSON URI conventions, and database dependencies.

5.5 Running the configurator

```
cd Exec/Rod/Studies
discharge-inception run RadiusStudy.py \
    --output-dir ~/my_rod_study \
    --dim 2 \
    --verbose
```

The configurator does four things:

1. Creates the output directory tree (`pdiv_database/`, `plasma_simulations/`, and all `run_N/` subdirectories).
2. Copies executables, input files, job scripts, and data files into place.
3. Submits a SLURM array job for the database (`pdiv_database/`).
4. Submits a second SLURM array job for the study (`plasma_simulations/`), chained to depend on the database job completing first.

See *The CLI* for all available options.

5.6 Output directory layout

Just after submission (before any SLURM job has run), the layout looks like:

```
$ ls -R --file-type ~/my_rod_study
.:
pdiv_database/  plasma_simulations/

./pdiv_database:
array_job_id  DischargeInceptionJobscript.py  ExtractElectronPositions.py
GenericArrayJob.sh  index.json  master.inputs  chemistry.json
electron_transport_data.dat  ion_transport_data.dat  detachment_rate.dat
main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex
run_0/  run_1/  run_2/  structure.json
```

(continues on next page)

(continued from previous page)

```
jobscript_symlink@

./pdiv_database/run_0:
master.inputs  parameters.json  chemistry.json
electron_transport_data.dat  ion_transport_data.dat  detachment_rate.dat
chk/  plt/  pout.*  main@

./plasma_simulations:
array_job_id  chemistry.json  GenericArrayJob.sh  GenericArrayJobJobscript.py
ExtractElectronPositions.py  inception_stepper@  index.json  master.inputs
PlasmaJobscript.py
main2d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex
run_0/  run_1/  run_2/  structure.json  jobscript_symlink@

./plasma_simulations/run_0:
chemistry.json  detachment_rate.dat  electron_transport_data.dat
ion_transport_data.dat  master.inputs  parameters.json  main@
```

Key points:

- `jobscript_symlink@` in `pdiv_database/` points to `DischargeInceptionJobscript.py`; in `plasma_simulations/` it points to `PlasmaJobscript.py`. `GenericArrayJob.sh` calls `python ./jobscript_symlink` without knowing which script it is.
- `plasma_simulations/inception_stepper@` is a symlink to `../pdiv_database`, giving the plasma study jobscript direct access to inception results. The symlink name matches the database `identifier` (`'inception_stepper'`).
- `run_N/main@` in both stages points to the executable in the parent stage directory. Both stages use the same binary.
- `master.inputs` in `pdiv_database/run_N/` has `app.mode = inception`; in `plasma_simulations/run_N/` it has `app.mode = plasma`. The configurator writes this automatically via `input_overrides`.

See *Output directory structure* for a full description of every metadata file.

5.7 Monitoring jobs

```
squeue -u $USER

# Check the submitted job IDs
cat ~/my_rod_study/pdiv_database/array_job_id       # database job ID
cat ~/my_rod_study/plasma_simulations/array_job_id  # study job ID (depends on above)
```

The study job will remain in `Pending` state (dependency not yet satisfied) until the database job finishes successfully.

5.8 Inspecting results

Use `discharge-inception ls` to see a table of runs with their parameter values and completion status:

```
discharge-inception ls ~/my_rod_study/pdiv_database/
```

Example output:

```
~/my_rod_study/pdiv_database  (3 runs)
  run     geometry_radius  pressure
  -------  ---------------  --------
  run_0    0.0001           100000  [done]
  run_1    0.0005           100000  [done]
  run_2    0.001            100000
```

The [done] mark indicates that `report.txt` is present in that run directory.

To inspect the exact parameters for a specific run:

```
cat ~/my_rod_study/pdiv_database/run_0/parameters.json
```

```
{
    "geometry_radius": 0.0001,
    "pressure": 100000.0
}
```

`index.json` at the stage level maps run indices to parameter tuples — useful for scripts that need to iterate all runs:

```
cat ~/my_rod_study/pdiv_database/index.json
```

```
{
    "prefix": "run_",
    "keys": ["geometry_radius", "pressure"],
    "index": {
        "0": [0.0001, 100000.0],
        "1": [0.0005, 100000.0],
        "2": [0.001, 100000.0]
    }
}
```

INDEX

Symbols

--format
    command line option, 26
--input
    command line option, 29
--lp
    command line option, 29
--lp-tau
    command line option, 29
--no-output
    command line option, 27
--output
    command line option, 26, 27, 29
--output-dir
    command line option, 31
--plot
    command line option, 26, 27
--prefix
    command line option, 31
--select
    command line option, 26
--sg
    command line option, 29
--sg-order
    command line option, 29
--sg-window
    command line option, 29
--tail
    command line option, 27
--voltage
    command line option, 26
-i
    command line option, 29
-o
    command line option, 29

C

command line option
    --format, 26
    --input, 29
    --lp, 29
    --lp-tau, 29
    --no-output, 27
    --output, 26, 27, 29
    --output-dir, 31
    --plot, 26, 27
    --prefix, 31
    --select, 26
    --sg, 29
    --sg-order, 29
    --sg-window, 29
    --tail, 27
    --voltage, 26
    -i, 29
    -o, 29
    db_dir, 26, 27, 31

D

db_dir
    command line option, 26, 27, 31